

Session 18 : Personnages Avancés — Ragdolls et Corps Mous

Objectifs de la session

- Comprendre le **ragdoll** : ce qui arrive quand le personnage **perd le contrôle** (mort, chute, impact).
- Construire un ragdoll : convertir un squelette en **corps rigides + joints** avec des limites d'angle.
- Introduire l'**active ragdoll** : mélanger animation et physique pour des mouvements hybrides.
- Couvrir les **corps mous** (soft bodies) : cheveux, capes, queues de cheval, chaînes — pilotés par des contraintes de distance.
- **Étudier** un vrai personnage : lire son squelette, décider du mapping ragdoll, puis **rigger et simuler** son ponytail.
- Voir comment ces techniques s'implémentent concrètement dans Blender et Rapier.

💡 De la Session 16 à ici

En Session 16, le personnage était **contrôlé** : le code décidait, le moteur vérifiait. Mais quand le personnage **meurt**, tombe d'une falaise, ou reçoit un coup de pied, le contrôle disparaît — c'est la physique qui reprend la main. C'est le **ragdoll**. Et quand le personnage a des éléments **mous** (cheveux, cape, queue de cheval, ceintures), ni le rigid body ni le KCC ne suffisent — il faut des **corps mous**.

Partie 1 : Le Ragdoll — quand le contrôle s'arrête

Ragdoll

Un **ragdoll** est un personnage simulé comme un **ensemble de corps rigides connectés par des joints**, sans aucun contrôle. Le moteur décide de tout : la gravité le fait tomber, les impulsions le projettent, les contraintes limitent les articulations.

Quand on l'utilise :

- Le personnage **meurt** ou est **stunned** — on veut qu'il s'affaisse naturellement.
- Le personnage reçoit un **impact** — explosion, coup, voiture.
- Le personnage **glisse** ou **tombe** dans un trou — la physique prend le dessus.

Ce n'est PAS : une animation. Le ragdoll **remplace** l'animation pendant la durée de la perte de contrôle.

Construction d'un ragdoll

Convertir un squelette en ragdoll = mapper chaque **os** à un **corps rigide** et chaque **articulation** à un **joint** :

Os	Corps rigide	Joint parent
Tête	Capsule ou sphère	Ball joint (cou)
Torse	Capsule ou boîte	Ball joint (taille)
Bras supérieur	Capsule	Ball joint (épaule)
Avant-bras	Capsule	Hinge (coude)
Cuisse	Capsule	Ball joint (hanche)

Os	Corps rigide	Joint parent
Tibia	Capsule	Hinge (genou)
Pied	Boîte	Ball joint (cheville)

Point critique : les **limites d'angle**. Sans elles, le ragdoll se tord dans tous les sens — bras à 360°, jambes à l'envers. Chaque joint doit avoir des **bornes** qui imitent l'amplitude humaine.

! Les limites d'angle — la clé du réalisme

Un ragdoll sans limites ressemble à un **spaghetti**. Les limites d'angle sont ce qui le fait ressembler à un **corps humain** :

- **Coude** : hinge joint, 150° d'amplitude (ne peut pas se plier en arrière).
- **Genou** : hinge joint, 135° (ne peut pas plier vers l'avant).
- **Épaule** : ball joint, cône de 90° (le bras peut tourner mais pas passer à travers le torse).
- **Hanche** : ball joint, cône de 45° (la jambe peut s'écarter mais pas à 180°).
- **Cou** : ball joint, cône de 45° (la tête ne peut pas regarder en arrière).

Sans ces limites, le ragdoll se contorsionne de manière non-physique — le coude passe à travers le corps, le pied se tord dans le mauvais sens.

Ragdoll dans Rapier.

```
// Créer un ragdoll à partir d'un squelette (Rapier 0.19)
const bones = extractSkeleton(mesh); // positions / orientations des os
const byName = {};

for (const bone of bones) {
    // 1. Un corps rigide par os. Tous les corps partagent la MÊME
    // orientation (l'identité) : l'axe d'un joint revolute est
    // alors valide pour le parent COMME pour l'enfant (cf. 8.6).
    const body = world.createRigidBody(
        RAPIER.RigidBodyDesc.dynamic()
        .setTranslation(bone.pos.x, bone.pos.y, bone.pos.z)
        .setRotation({ x: 0, y: 0, z: 0, w: 1 })
    );
    // 2. C'est le COLLIDER qui porte la rotation de l'os.
    world.createCollider(
        RAPIER.ColliderDesc.capsule(bone.halfHeight, bone.radius)
        .setRotation(bone.quat)
        .setDensity(bone.density),
        body // <- le corps parent
    );
    byName[bone.name] = { body, bone };
}

// 3. Un joint par articulation.
for (const bone of bones) {
    if (!bone.parent) continue;
    const p = byName[bone.parent], c = byName[bone.name];
    const anchor = {
        x: c.bone.pos.x - p.bone.pos.x,
        y: c.bone.pos.y - p.bone.pos.y,
        z: c.bone.pos.z - p.bone.pos.z };
}
```

```

let joint;
if (bone.hingeAxis) {
    // Hinge = revolute + butées. Dans cette version c'est le
    // SEUL joint qui accepte des limites d'angle.
    joint = world.createImpulseJoint(
        RAPIER.JointData.revolute(anchor, { x: 0, y: 0, z: 0 }, bone.hingeAxis),
        p.body, c.body, true);
    joint.setLimits(0, bone.maxAngle);
} else {
    // Ball = sphérique, 3 ddl – mais AUCUNE limite d'angle.
    joint = world.createImpulseJoint(
        RAPIER.JointData.spherical(anchor, { x: 0, y: 0, z: 0 }),
        p.body, c.body, true);
}
joint.setContactsEnabled(false); // évite le jitter entre os voisins
}

```

⚠ Ce que l'API ne permet pas (Rapier 0.19)

Attention aux exemples qu'on trouve en ligne : JointData n'a **pas** de méthode ball, et un joint **sphérique** n'accepte **ni limites d'angle ni moteur** — limitsEnabled() est un simple lecteur, il n'existe pas de setter. Les noms réels sont spherical, revolute, prismatic, fixed, rope, spring et generic.

C'est une contrainte forte : un ragdoll dont les épaules et les hanches sont des joints sphériques est un tas de spaghettis qu'aucune correction externe ne tient. C'est pour ça que la démo de cette session finit par tout monter en revolute (cf. 8.6).

Partie 2 : Active Ragdoll — le meilleur des deux mondes

Active ragdoll

Un **active ragdoll** est un ragdoll qui **essaie** de rester dans une pose cible. Au lieu d'être complètement mou, il **lutte** — les joints appliquent des **moteurs** (moteurs physiques, vus en S15) pour se rapprocher de la pose animée.

Résultat : un personnage qui **résiste** aux impacts sans être complètement rigide. Exemples :

- Un zombie qui **tente** de rester debout après un coup.
- Un personnage qui **protège** sa tête quand il tombe.
- Une créature qui **se balance** sur un fil.

Principe

À chaque frame :

1. On **mesure** la pose actuelle du ragdoll (angles de chaque joint).
2. On **compare** à la pose cible (l'animation qui jouerait si le ragdoll était désactivé).
3. On **applique** un **moteur** sur chaque joint pour rapprocher la pose actuelle de la pose cible.
4. Le moteur est un **PD controller** — proportionnel à l'erreur angulaire + amorti par la vitesse angulaire.

$$\tau = k_p \cdot (\theta_{\text{cible}} - \theta_{\text{actuel}}) - k_d \cdot \omega_{\text{actuel}}$$

k_p est la **raideur** (combien le joint “tire” vers la cible), k_d est l’**amortissement** (combien le joint “freine” les oscillations).

💡 Le mixte : animation + ragdoll

En pratique, on ne bascule pas brutalement entre KCC et ragdoll — on **mélange** :

- **Impact léger** : le ragdoll absorbe l’onde mais l’animation reprend le dessus.
- **Impact fort** : le ragdoll prend le dessus, le personnage tombe.
- **Rétablissement** : l’active ragdoll ramène progressivement la pose vers l’animation, puis le KCC reprend.

Le mélange se fait par un **poids** α : $\alpha = 0 \rightarrow$ full animation (KCC), $\alpha = 1 \rightarrow$ full ragdoll. On fait varier α selon l’intensité de l’impact.

Partie 3 : Corps Mous — les éléments déformables

💡 Quand le rigid body ne suffit plus

Le rigid body est **rigide** — il ne se déforme jamais. Mais les personnages ont des parties **molles** qui bougent avec le corps :

- **Cheveux / queue de cheval** : ondulent avec la marche, rebondissent après un saut.
- **Cape / manteau** : flotte dans le vent, pend au repos.
- **Ceinture / sac** : oscille avec le mouvement.
- **Écailles / carapace** : se déforment sous l’impact.

Ces éléments ne sont **ni rigides ni liquides** — ils sont **élastiques** : ils se déforment mais **rappellent** leur forme originale.

Modèle de corps mou

Un corps mou est un **réseau de points** (vertices) connectés par des **contraintes de distance** :

- Chaque point a une **position** et une **vitesse** (pas de masse rigide).
- Chaque contrainte impose une **distance** entre deux points (comme un ressort).
- Le tout est simulé par **Verlet** ou **PBD** (Session 8) — pas par des rigid bodies.

La différence clé avec un ragdoll : les corps mous n’ont pas de **joints à angles** — ils ont des **ressorts** (contraintes de distance). Plus simple, plus rapide, parfait pour les éléments qui **pendent** ou **ondulent**.

Partie 4 : PBD pour les corps mous — le principe

Contrainte de distance

La contrainte de base d’un corps mou est la **contrainte de distance** : deux points p_1 et p_2 doivent rester à une distance d l’un de l’autre.

```
// Résolution d'une contrainte de distance (PBD)
function solveDistance(p1, p2, restDist, stiffness = 1.0) {
    const diff = p2.sub(p1);
    const dist = diff.length();
    if (dist === 0) return;
```

```

// Erreur : distance actuelle vs distance cible
const error = dist - restDist;
// Correction : déplacer les deux points pour réduire l'erreur
const correction = diff.multiplyScalar(
  (error / dist) * stiffness * 0.5
);
p1.add(correction);
p2.sub(correction);
}

```

p_1 est tiré vers p_2 , p_2 est tiré vers p_1 — les deux se rapprochent ou s'éloignent pour respecter la distance. stiffness contrôle combien de la correction est appliquée par itération (1.0 = rigide, 0.1 = élastique).

La chaîne — le cas du ponytail

Une queue de cheval, une mèche de cheveux, ou une cape est une **chaîne** de points connectés par des contraintes de distance :

```

// Chaîne de N points (ex: 5 points = 4 segments pour une queue de cheval)
const chain = {
  points: [
    { pos: attach, prev: attach, pinned: true }, // point 0 : attaché au
    crâne
    { pos: attach + (0, -0.2, 0), prev: ..., pinned: false },
    { pos: attach + (0, -0.4, 0), prev: ..., pinned: false },
    { pos: attach + (0, -0.6, 0), prev: ..., pinned: false },
    { pos: attach + (0, -0.8, 0), prev: ..., pinned: false },
  ],
  segmentLength: 0.2, // distance entre points
  iterations: 4 // itérations de résolution par frame
};

// Simulation : Verlet + contraintes
function simulateChain(chain, gravity, dt) {
  // 1. Intégration Verlet (Session 8)
  for (const p of chain.points) {
    if (p.pinned) continue;
    const vel = p.pos.sub(p.prev);
    p.prev = p.pos;
    p.pos = p.pos.add(vel.multiplyScalar(0.99)) // friction de l'air
      .add(gravity.multiplyScalar(dt * dt));
  }

  // 2. Résolution des contraintes (plusieurs itérations)
  for (let i = 0; i < chain.iterations; i++) {
    for (let j = 0; j < chain.points.length - 1; j++) {
      solveDistance(chain.points[j], chain.points[j + 1],
        chain.segmentLength, 0.8);
    }
    // Contrainte de raideur : empêcher la chaîne de se replier trop
    // (optionnel — la queue de cheval n'est pas complètement flasque)
  }
}

```

Le premier point est **pinné** (attaché au crâne du personnage) — il suit le personnage. Les autres points **pendent** sous la gravité et suivent par inertie.

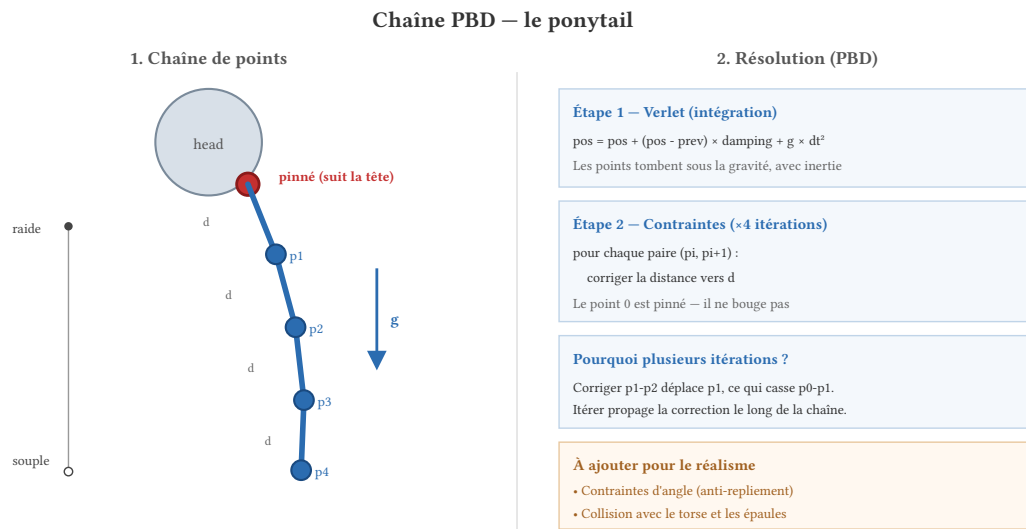


Figure 1: Le ponytail comme chaîne PBD. À gauche : la chaîne de points, avec le point 0 pinné sur la tête (en rouge) et les segments de longueur d — la raideur décroît de la base vers la pointe. À droite : la boucle de résolution — intégration Verlet, puis plusieurs itérations de contraintes de distance.

💡 La queue de cheval en pratique

Une queue de cheval n'est **pas** une chaîne libre — elle est **raide** à la base (attachée au crâne) et **souple** à l'extrémité (les pointes). Pour simuler ça :

- **Contraintes de raideur** : des contraintes de distance plus fortes près de la base (la racine bouge moins).
- **Contraintes d'angle** : limiter l'angle maximum entre segments adjacents (la queue ne peut pas se replier à 180° sur elle-même).
- **Damping** : la friction de l'air ralentit les pointes (elles oscillent moins).
- **Collision** : la queue ne doit pas traverser le cou ou les épaules — ajouter des colliders sur le torse et la tête.

Partie 5 : Autres corps mous

Cape et manteau — le tissu 2D

Une cape est un **maillage 2D** de points connectés par des contraintes de distance horizontales et verticales. Chaque point est lié à ses 4 voisins (haut, bas, gauche, droite) — comme un drap suspendu.

Pour éviter le **stretching** excessif, on ajoute des contraintes **diagonales** (shear constraints) qui empêchent le tissu de se déformer en losange. C'est exactement le **cloth** de la Session 8, mais appliqué à un personnage au lieu d'un drapeau.

Gelée / blob — le corps mou 3D

Un blob (blobfish, slimes, gelée) est un **réseau 3D** de points avec des contraintes de distance dans toutes les directions. Pour conserver le **volume** (le blob ne doit pas s'écraser complètement), on ajoute une **contrainte de volume** :

$$V_{\text{actuel}} = \sum_i \text{volume}(\text{tétraèdre}_i)$$

On compare V_{actuel} à V_{repos} et on applique une force de **pression** qui gonfle ou dégonfle le blob pour restaurer le volume. C'est le principe des **pressure soft bodies** — utilisé dans les jeux pour les objets rebondissants (ballons, jelly, pneus).

Ceintures, sacs, accessoires

Un accessoire qui **pend** d'un point d'attache (ceinture, sac à dos, harnais) est une **chaîne courte** (2-3 segments) avec une masse au bout. C'est le cas le plus simple — le même principe que le ponytail, mais plus court et plus lourd.

Ces éléments ajoutent énormément de **vie** à un personnage — ils rendent les mouvements **crédibles** sans aucune animation manuelle.

Partie 6 : Cas pratique — le personnage complet

! L'architecture complète

Un personnage de jeu moderne combine **tous** les niveaux de la hiérarchie physique :

Élément	Type	Technique
Déplacement	Contrôlé	KCC (S16) — capsule, move_and_slide
Animation	Contrôlé	FK (S16) — angles prédéfinis
Articulations	Hybride	IK (S16) — foot placement, look-at
Mort/impact	Non contrôlé	Ragdoll — rigid bodies + joints
Résistance	Hybride	Active ragdoll — moteurs PD
Cheveux	Non contrôlé	Chaîne PBD — contraintes de distance
Cape	Non contrôlé	Cloth 2D — maillage de contraintes
Blob/gelée	Non contrôlé	Soft body 3D — contraintes de volume

La **clef** : ces systèmes coexistent. Le personnage marche (KCC), ses cheveux ondulent (PBD), son épée pend (contrainte), et quand il meurt — le tout bascule en ragdoll.

Pipeline de transition.

```
// Transition contrôlé → ragdoll → contrôlé
function onImpact(force) {
    if (force > DEATH_THRESHOLD) {
        // Passer en mode ragdoll
        characterController.enabled = false;
        ragdoll.activate();
        // Copier la vélocité du KCC vers le ragdoll
    }
}
```

```

        ragdoll.setVelocities(controller.velocity);
    }
}

function update(dt) {
    if (ragdoll.active) {
        // Vérifier si le ragdoll s'est stabilisé
        if (ragdoll.settled() && canGetUp()) {
            // Transition retour : IK pour se relever
            ragdoll.blendToAnimation(standUpPose, 0.5); // 0.5s
            if (blendComplete()) {
                ragdoll.deactivate();
                characterController.enabled = true;
            }
        }
    }
}
}

```

Partie 7 : Blender et les outils

💡 Blender — simulation de corps mous

Blender a récemment amélioré son système de **soft body physics** :

- **Cloth** : simulation de tissu avec contraintes de distance, pinning, et collision avec le corps.
- **Soft Body** : déformation volumique pour les objets élastiques (jelly, gelée, balles).
- **Hair** : les particules de cheveux peuvent être simulées avec des contraintes de distance — exactement le même principe que le ponytail.
- **Rigid Body** : pour les accessoires rigides attachés au personnage.

Le **workflow** : on anime le personnage en FK/IK, puis on **bake** la simulation des corps mous par-dessus. Les résultats sont exportables en FBX/GLTF pour les jeux.

💡 Moteurs de jeu

- **Rapier** : `ImpulseJoint` pour les ragdolls, `RigidBodyDesc.dynamic()` pour les corps. Pas de soft body natif — on les simule avec PBD custom.
- **PhysX** (Unreal) : `PxArticulation` pour les ragdolls, `NvCloth` pour le tissu.
- **Havok** : le standard AAA pour les ragdolls et les corps mous.
- **Unity** : `ArticulationBody` pour les ragdolls, `Cloth component` pour les tissus.
- **Godot** : `PhysicalBoneSimulator3D` pour les ragdolls, `SoftBody3D` pour les corps mous.

Partie 8 : Démonstration — notre personnage dans Blender

💡 Objectif de la démo

Au lieu d'un TP, on **étudie ensemble** un personnage réel : on **lit** son squelette, on **décide** du mapping ragdoll, on **rigge** son ponytail, puis on observe ce que la physique en fait — d'abord en temps réel dans une démo navigateur, ensuite dans Blender.

8.1 Le personnage

Ce qu'on a

Un personnage généré (Tripo) — environ **1 m de haut**, 5 043 vertices, 5 555 polygones, un seul matériau. Le mesh est déjà **skinné** : il possède un modificateur Armature et **60 vertex groups**. (Le squelette compte 61 os, mais root n'a pas de groupe — c'est un os de service.)

Le squelette suit la **convention Unreal Engine 5** — c'est le standard de l'industrie, reconnaissable immédiatement :

- root → pelvis → spine_01 → spine_02 → spine_03 → neck_01 → head
- Bras : clavicle → upperarm → lowerarm → hand → doigts
- Jambes : thigh → calf → foot → ball

🧐 Pourquoi la convention UE5 est une bonne nouvelle

Parce que le squelette est **documenté** et **universel**. Les limites d'angle qu'on va choisir pour le ragdoll sont les mêmes que celles d'Unreal, de Unity, ou de n'importe quel asset store. On n'invente rien — on applique des valeurs connues. Et si on change de personnage plus tard, le code du ragdoll reste **identique**.

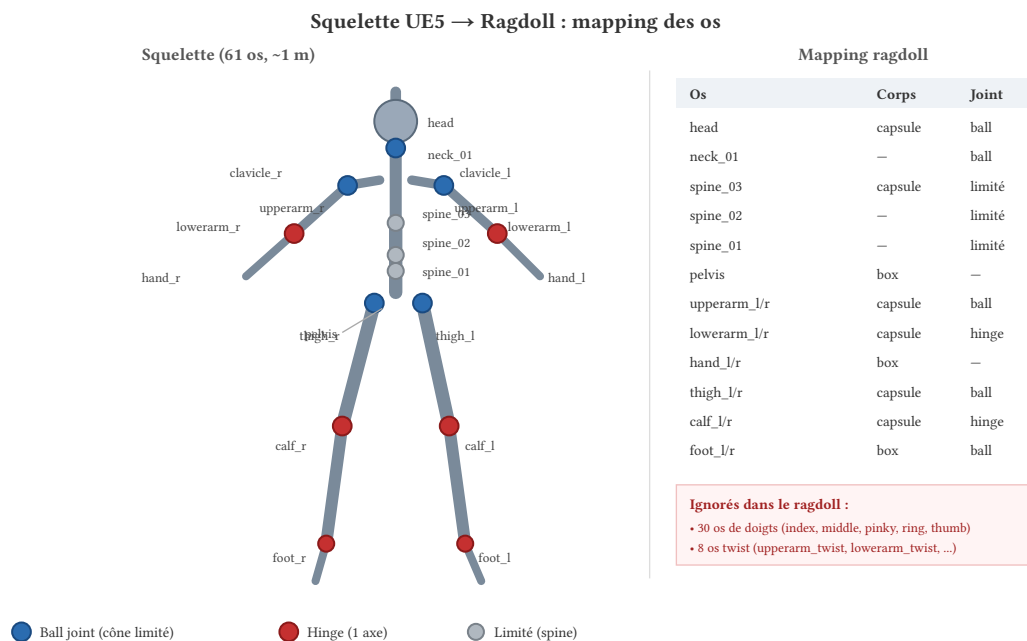


Figure 2: Notre personnage : le squelette UE5 (à gauche) et le mapping ragdoll **anatomique** (à droite). En bleu les articulations à 3 ddl (cou, épaules, hanches), en rouge celles à un seul axe (coudes, genoux, chevilles), en gris la colonne. Les os de doigts et les os **twist** sont ignorés dans le ragdoll.

C'est le **design** visé — la démo, elle, simplifie tout en joints revolute (cf. 8.6).

8.2 Lecture du squelette — l'exercice

Compter les os et décider ce qu'on garde

61 os, c'est **trop** pour un ragdoll. Chaque os = un rigid body = du coût CPU et de l'instabilité. On va **simplifier** :

Catégorie	Nombre	Décision
Spine + tête + cou	6	Gardés — c'est le tronc
Bras (clavicle→hand)	8	Gardés — mais clavicle fusionné au spine
Jambes (thigh→ball)	8	Gardés — ball fusionné au foot
Doigts	30	Ignorés — les mains restent des boîtes rigides
Os twist	8	Fusionnés à leur parent
root	1	Ignoré — c'est un repère, pas un corps

Résultat : **61 os → 22 corps rigides**. C'est le bon compromis pour un personnage de jeu — assez détaillé pour être crédible, assez léger pour en avoir 20 à l'écran.

⚠ Les os twist — le piège classique

Le squelette contient 8 os **twist** (upperarm_twist_01_l, lowerarm_twist_01_r, thigh_twist_01_l, ...). Ce sont des os **de skinning** : ils n'existent que pour améliorer la déformation du mesh quand le bras tourne. Ils n'ont **aucune** fonction articulaire.

Si on leur donne un rigid body, on crée des collisions **fantômes** et des contraintes **inutiles** — le ragdoll devient instable et tremble. Il faut les **fusionner** à leur parent (même corps rigide, pas de joint). C'est le premier réflexe à avoir face à un squelette de production.

Second piège, plus sournois : les twist bones cassent aussi la **mesure** de la longueur des os. Si on prend « le premier enfant » d'un os pour en déduire sa longueur, on tombe parfois sur le twist bone au lieu de l'os de la chaîne — et l'ordre des enfants n'est même pas symétrique. Sur notre personnage, upperarm_l liste upperarm_twist_01_l en premier (à 2 mm de son parent) alors que upperarm_r liste lowerarm_r : le bras gauche recevait une capsule de 3 cm et le droit une de 23 cm. La parade : viser l'enfant qui appartient à la **définition du ragdoll**, jamais « le premier venu ».

8.3 Choisir les limites d'angle

Les valeurs du ragdoll

Pour chaque joint, on définit une **amplitude**. Ces valeurs viennent de l'anatomie humaine — elles sont standardisées dans tous les moteurs :

Joint	Type	Axe / cône	Amplitude
Cou	Ball	Cône	$\pm 45^\circ$ avant/arrière, $\pm 60^\circ$ côté
Épaule	Ball	Cône	$\pm 90^\circ$ — le bras ne traverse pas le torse
Coude	Hinge	1 axe	0° à 150° — jamais en arrière
Poignet	Ball	Cône	$\pm 70^\circ$
Hanche	Ball	Cône	$\pm 45^\circ$ écart, -30° à $+120^\circ$ flexion

Joint	Type	Axe / cône	Amplitude
Genou	Hinge	1 axe	0° à 135° — jamais vers l'avant
Cheville	Hinge	1 axe	±40°
Spine	Limité	3 axes	±20° par segment

Attention : cette table est la CIBLE, pas le code. Elle décrit ce qu'utilise un ragdoll de **production** : des **ball joints** (3 ddl) pour les épaules, hanches, cou et poignets, des **hinges** (1 ddl) pour les coudes et genoux. La démo, elle, simplifie — **toutes** ses articulations sont des **revolute** (1 ddl), avec des bornes asymétriques. On perd les 3 ddl d'une épaule, mais on gagne des limites et un tonus que le solveur tient réellement (cf. 8.6).

⚠ Le sens de « positif » dépend du sens de l'os

Piège classique : deux articulations qui partagent le **même axe** n'ont pas forcément le même sens positif. Sur ce personnage, l'axe est X (gauche-droite) et il regarde vers **+Z** :

- un os qui **monte** (colonne, cou) : + penche vers l'**avant** ;
- un os qui **descend** (cuisse, tibia) : + part vers l'**arrière**.

D'où des bornes qui semblent incohérentes entre elles : la colonne est limitée à $[-5^\circ, +20^\circ]$ — on se plie en avant, très peu en arrière — alors que la hanche est à $[-90^\circ, +15^\circ]$. Une fois le sens connu, tout redevient cohérent.

Ça se mesure, ça ne se devine pas : on lit la direction des orteils (foot → ball) pour trouver l'avant du personnage, puis on applique une rotation de $+20^\circ$ autour de l'axe et on regarde de quel côté l'os bascule.

💡 Le test visuel

Pour valider les limites : on **tire** le ragdoll par le bras et on regarde ce qui se passe. Les erreurs typiques se voient tout de suite :

- **Coude qui se plie en arrière** → la limite du coude est trop large (ou absente).
- **Genou qui part vers l'avant** → le hinge est monté sur le mauvais axe.
- **Tête qui rentre dans le torse** → le cône du cou est trop grand.
- **Bras qui traverse la poitrine** → le cône de l'épaule est trop grand.

C'est un réglage **itératif** — on ajuste, on relance, on regarde. Comptez une dizaine d'essais avant que ce soit convaincant.

8.4 Le ponytail — rig et chaîne PBD

💡 État initial : le ponytail n'était PAS riggé

Le mesh contient une **queue de cheval** dans sa géométrie, mais elle n'avait **aucun os** : elle était skinnée sur head comme le reste du crâne. En ragdoll, elle restait donc **collée à la tête**, parfaitement rigide. On a mesuré : sur 5043 vertices, **451** lui appartiennent, et ils étaient tous à 100 % sur head.

Ce qu'on a fait — le rig

1. **Chaîne d'os** créée le long de l'axe de la queue : pony_01 → pony_02 → pony_03 → pony_04. Longueur d'arc totale 0.33 unités Blender (≈ 56 cm à l'échelle de la démo), soit 14 cm par segment.
2. **Premier os reparenté à head** (Connected désactivé : la queue ne touche pas le crâne, elle doit garder son décalage).
3. **Transfert de poids** : les vertices de la queue sont sortis de head et répartis sur la chaîne. Après transfert, head passe de 6985 à 4111 vertices — et $6985 - 4111 = 2874$, exactement le nombre de vertices du ponytail après découpage par le FBX.
4. **Vérification en FK** : en tournant pony_02 de 40° , la pointe de l'os bouge de 0.200 — et les vertices de la pointe suivent à **101 %**, la racine restant à 0.0 (elle reste soudée au crâne).

Deux options pour la simulation

Une fois la chaîne en place, deux chemins :

Option A — PBD temps réel (moteur de jeu)

- La chaîne devient une chaîne de **points** avec contraintes de distance (Partie 4).
- Le premier point est **pinné** sur head (il suit la tête).
- Simulation à 60 Hz, 4 itérations de contraintes par frame.
- **Avantage** : temps réel, réagit à tout (vent, explosions, collisions).
- **Inconvénient** : approximation — peut traverser les épaules si mal réglé.

Option B — Soft body Blender (bake offline)

- La queue est simulée en **soft body** / **cloth** dans Blender avec collision sur le corps.
- La simulation est **bakée** sur une animation (idle, marche, saut).
- Le résultat est exporté en FBX/GLTF comme une animation classique.
- **Avantage** : haute fidélité, collisions précises, aucune limite de coût.
- **Inconvénient** : figé — ne réagit pas aux événements imprévus du jeu.

💡 Option A implémentée — la chaîne PBD de la démo

C'est l'option A qui tourne dans `session18_ragdoll.js`. Concrètement, 5 points et 4 segments :

- Le point 0 est **épinglé** sur head : à chaque frame on le remplace sur l'os, donc la queue suit la tête.
- Les points 1 à 4 sont intégrés en **Verlet** (position précédente + gravité), puis **8 itérations** de contraintes de distance rétablissent les longueurs de repos.
- Une **raideur de repli** limite le pliage : contrainte de distance entre les points espacés de **deux** (i et $i+2$). À 100 % la chaîne est rigide, à 0 % elle est libre.
- Une **tenue de racine** rappelle le premier segment vers sa position de repos dans le repère de la tête — sans elle la queue tombe trop à la racine et vient trop près du dos.

Deux pièges rencontrés, qui valent la peine d'être connus :

- Le premier essai utilisait un **shape matching** vers la pose de repos. La correction y est proportionnelle à l'**écart** — et c'est le bout de la queue qui s'écarte le plus. Résultat : le bout était plus raide que la racine. La contrainte de distance entre i et $i+2$ traite au contraire **chaque triplet identiquement**, donc la raideur est uniforme.

- L'amortissement était appliqué **par frame** (0.94) et non par seconde : la vitesse tombait à 2 % en une seconde, donc la queue ne bougeait presque pas, quelle que soit la raideur. À 60 Hz il faut 0.985.

Les os pony_* ne sont **pas** dans la table RAGDOLL : le ragdoll ne les pilote pas. On les simule à part, puis on écrit leur orientation à partir de la chaîne — le maillage suit tout seul, puisque les poids sont déjà dessus.

💡 L'approche hybride (celle des AAA)

En production, on combine les deux :

- Les **animations de base** (idle, marche, course) ont la queue **bakée** par Blender — qualité maximale, coût zéro à l'exécution.
- Les **événements dynamiques** (ragdoll, explosion, vent fort) basculent sur la simulation **temps réel** PBD.
- La transition se fait par un **blend** sur 0.3s pour éviter le pop visuel.

C'est exactement la même philosophie que le ragdoll vs l'animation (Partie 2) : le **baked** pour la qualité, le **temps réel** pour la réactivité, et un **mélange** entre les deux.

8.5 Ce qu'on va observer

Déroulé de la démo

1. **Lire le squelette** dans Blender — compter les os, repérer les twist et les doigts.
2. **Repérer les articulations** — identifier visuellement quels joints sont des hinges (coudes, genoux) et lesquels sont des balls (épaules, hanches).
3. **Simuler le ragdoll** — le faire tomber, observer les limites, ajuster.
4. **Rigger le ponytail** — lui donner sa chaîne de 4 os et transférer les poids (cf. 8.4).
5. **Discuter** du plan de rigging et du choix PBD vs bake.

8.6 La démo navigateur

💡 `examples/session18\_ragdoll.html`

Le **même personnage**, chargé depuis le FBX et converti en ragdoll dans **Rapier** (Three.js + WASM). C'est la version exécutable de tout ce qu'on vient de voir — et le meilleur moyen de **sentir** l'effet des limites et de la raideur articulaire.

Le personnage est skinné, le FBX est chargé par FBXLoader, et les textures PBR sont appliquées à la main (le FBX de Tripo référence des chemins absolus qui n'existent pas ici).

Le ragdoll : 18 corps rigides (une capsule par os) et 17 articulations. Chaque corps est placé exactement sur la transformation monde de repos de son os — la synchro corps → os est donc une simple copie, sans décalage à gérer.

Pourquoi **toutes** les articulations sont des **revolute** (1 ddl) et pas des **sphériques** pour les épaules et les hanches ? Parce que dans Rapier 0.19 les joints sphériques n'ont **ni limites d'angle ni moteur** : un ragdoll monté ainsi est un tas de spaghettis impossible à tenir, et aucune de nos corrections externes n'y changeait rien. Les joints revolute, eux, exposent `setLimits` et `configureMotorPosition` — deux contraintes résolues **par le solveur**, donc impossibles à

annuler. On perd les 3 ddl d'une épaule, mais on gagne des limites et un tonus qui tiennent vraiment.

La physique avance à **pas fixe** (1/60 s, accumulé) : avec un dt variable, les contraintes ne convergent pas de la même façon d'une frame à l'autre et le ragdoll tremble.

Trois modes (GUI) :

- **Pose** — le personnage est figé dans sa pose de repos. C'est l'équivalent du **character controller** de la Session 16 : le code décide, rien ne bouge tout seul.
- **Ragdoll** — la physique prend le dessus. Le personnage s'effondre.
- **Active ragdoll** — les mêmes moteurs, mais beaucoup plus raides : le personnage **résiste** et essaie de revenir à sa pose de repos.

Ce qu'il faut manipuler :

- **Raideur k_p** : c'est le **tonus musculaire**. À 0, le ragdoll est un tas de spaghettis ; vers 60, il se tient ; au-delà, il devient rigide. C'est la démonstration la plus parlante de la session. Détail utile : le moteur de Rapier est en modèle **accélération**, donc k_p s'exprime en $1/s^2$ (soit ω^2 , une fréquence au carré) — la raideur est donc **indépendante de la masse** du corps, ce qui évite qu'un petit corps (la tête) parte en vrille.
- **Butées d'angle** : les désactiver autorise les hyperextensions — le coude se plie à l'envers et le genou part dans le mauvais sens.
- **Frottement sol (μ)** : à 1.0 les pieds accrochent et le personnage bascule ; vers 0 il glisse et s'affaisse. Piège de l'API : la règle de combinaison est en **Max**, donc le frottement effectif d'un contact vaut $\max(\text{sol}, \text{corps})$ — baisser le sol **seul** ne change rien, il faut baisser les deux.
- **Corps / Joints** : afficher les capsules et les lignes de joint pour voir le mapping os $\rightarrow$ corps rigide.
- **Ponytail** : la chaîne PBD. **Raideur** = résistance au pliage, **Tenue racine** = à quel point le premier segment suit la tête, **Gravité** / **Amorti** / **Itérations** comme dans la Partie 4.
- **Espace** : pousser le ragdoll. La direction est **tirée au hasard** (azimut uniforme), donc deux poussées ne partent jamais dans le même sens. **R** : reset.

Le détail qui compte : les corps du ragdoll ne collisionnent **pas entre eux** (groupes de collision) — seulement avec le sol et les obstacles. Sinon les capsules voisines (les deux cuisses, par exemple) se repoussent en permanence et le ragdoll tremble.

⚠ Piège 1 — l'axe des joints Rapier

`JointData.revolute(a1, a2, axe)` ne prend qu'**un seul** axe — et cet axe est interprété dans l'espace **local des deux corps** à la fois. Si on le calcule dans le repère de l'enfant, il devient faux pour le parent : le solveur applique alors une impulsion colossale pour aligner deux axes qui ne peuvent pas l'être, et le ragdoll explose.

La parade : donner à **tous** les corps la **même** orientation (l'identité) et porter la rotation de l'os sur le **collider**. Un axe exprimé en espace monde est alors valide pour le parent comme pour l'enfant, sans aucun calcul de repère. (Les versions plus récentes de `rapier.js` exposent `revoluteWithAxes`, mais pas la 0.19.3 utilisée ici.)

D'où vient l'axe d'un joint ?

Puisque les corps sont tous en orientation identité, l'axe d'un joint s'exprime directement en **espace monde** — il n'y a aucun repère à convertir. Trois cas :

- side, up, fwd → simplement X, Y ou Z du monde.
- limb → **dérivé** : `cross(direction de l'os, up)`.

Le cas limb cache une subtilité : pour un os **vertical** (colonne, cuisse, tibia), `cross(-Y, +Y)` vaut **zéro** — l'axe dégénère et Rapier refuse un axe non normalisable. On retombe alors sur l'axe X.

Ce n'est pas cosmétique : c'est ce qui fait que les bras et les jambes ne se plient pas dans le même plan.

- Un bras pointe vers $\pm X$, donc `cross(bras, up)` donne $\pm Z$ → l'épaule et le coude tournent dans le plan perpendiculaire à Z.
- Une cuisse pointe vers le bas, donc on retombe sur X → la hanche et le genou tournent dans le plan **sagittal**, celui dans lequel on marche.

⚠ Piège 2 — l'échelle d'un squelette skinné

Deux erreurs classiques quand on pilote un squelette skinné depuis la physique :

- Reconstruire la matrice monde de l'os en forçant l'échelle à 1 écrase l'échelle du modèle (ici $\times 1.7$). La matrice de skinning `bone.matrixWorld × boneInverse` perd son facteur d'échelle et le mesh se rétracte vers l'origine de chaque os : le personnage paraît **étiré** et squelettique. Il faut mémoriser et réappliquer l'échelle **monde** de repos de chaque os.
- Mesurer la longueur des os avec la position **locale** de l'enfant (`c.position.length()`) donne une valeur non mise à l'échelle — les capsules physiques sont alors fausses d'un facteur `modelScale`. Il faut mesurer en **espace monde**.

⚠ Piège 3 — les itérations du solveur

Un ragdoll n'est pas une collection d'objets isolés : c'est une **chaîne profonde** de contraintes (pelvis → colonne → épaules → coudes → mains). Le solveur les résout par itérations successives, et chaque articulation dépend de ses voisines — tant que le bas de la chaîne n'a pas convergé, le haut n'a pas de solution stable.

Avec les **4 itérations par défaut** de Rapier, la convergence n'est pas atteinte et les ancrs restent violées : on voit alors le bassin se « déboîter ». Mesuré sur notre personnage, l'écart d'ancre monte à **10,8 cm**. À **16 itérations**, il tombe sous le centimètre — sans toucher aux masses ni aux limites.

C'est le réflexe à avoir : quand un ragdoll paraît « mou » ou « désarticulé » alors que ses limites sont bonnes, on ne bricole pas les masses — on commence par **laisser le solveur converger**.

Second effet, réel mais secondaire : l'os pelvis d'un squelette UE5 ne mesure que 6 cm. Tel quel, il donne un corps **29 fois plus léger** que chaque cuisse, et le solveur a du mal à transmettre les forces entre le torse et les jambes. On lui impose donc des dimensions de bassin humain, proportionnelles à la taille du personnage.

💡 Ce qu'il faut retenir

- Un squelette de **production** n'est jamais un squelette de **ragdoll** — il faut simplifier (doigts, twist) et ajouter des limites.
- Les **limites d'angle** sont ce qui distingue un ragdoll crédible d'un tas de spaghettis.
- La **raideur articulaire** (le tonus) doit être appliquée par le **solveur** — un moteur de joint, pas un couple ajouté à la main. Un couple externe est réabsorbé par la résolution des contraintes, donc il ne « tient » pas.
- Le **sens** d'une limite se **mesure** (direction des orteils + une rotation test), il ne se devine pas : un même axe tourne dans deux sens selon que l'os monte ou descend.
- Les **corps mous** (cheveux, capes) ne sont pas des rigid bodies — ce sont des **chaînes de contraintes de distance**, exactement le PBD de la Session 8.
- En production, on **mélange** toujours : baked pour la qualité, temps réel pour la réactivité.